Day 9: Repeat Menu Using While Loop

1. Day Number

Day 9

2. Topic Name

While Loop and `break`

Today you will learn how to:

- Repeat a block of code using `while`
- Stop a loop using `break`
- Keep showing a menu until the user chooses to exit

---

3. Connection with Yesterday

Yesterday, you created a simple menu that appeared only once:

```
1. Check Balance
2. Exit
```

The user selected one option, the program printed a message, and then the program ended.

Today, you will improve that program.

The menu will keep appearing again and again until the user selects:

```
2. Exit
```

---

4. Important Topics

`while`

A `while` loop repeats code as long as its condition is `True`.

```
while condition:
    # repeated code
```

`break`

`break` immediately stops the loop.

```
if user_wants_to_exit:
    break
```

Menu loop

A menu loop repeatedly:

1. Displays the options
2. Takes the user's choice
3. Performs an action

4. Displays the menu again

Condition

A condition is something that Python checks as either:

```
True
```

or:

```
False
```

---

5. Foundational Notes

What is a loop?

A loop allows us to repeat code without writing the same code many times.

Without a loop:

```
print("Menu")
print("Menu")
print("Menu")
```

With a loop:

```
while some_condition:
    print("Menu")
```

What is an infinite loop?

An infinite loop keeps running because its condition never becomes `False`.

For menu programs, an infinite loop can be useful:

```
while True:
    print("Show menu")
```

Here, `True` always remains true, so the loop keeps running.

We stop it using `break`.

How `break` works

```
while True:
    choice = input("Enter your choice: ")

    if choice == "2":
        break
```

When the user enters `"2"`, Python reaches `break` and exits the loop.

Why compare the choice as a string?

The `input()` function returns text.

Therefore, this is valid:

```
choice == "1"
```

You do not necessarily need `int()` for a simple menu.

Comparing strings can also prevent the program from crashing when the user enters text such as:

```
hello
```

---

6. Easy Example

This example repeatedly asks the user whether they want to continue:

```
while True:
    answer = input("Enter yes to continue or no to stop: ")

    if answer == "no":
        print("Program stopped")
        break

    print("Program is continuing")
```

How it works

- `while True` starts a repeating loop.
- The user enters an answer.
- If the answer is `"no"`, `break` stops the loop.
- Otherwise, the loop continues.

This is only an example of `while` and `break`, not the solution to today's problem.

---

7. Problem Statement

Create a simple bank menu.

The program should repeatedly display:

```
1. Check Balance
2. Exit
```

Requirements:

- Create a balance variable.
- Keep showing the menu.
- Ask the user to select an option.
- If the user selects `1`, display the current balance.
- If the user selects `2`, display an exit message and stop the loop.
- For any other choice, display an invalid-choice message.
- After checking the balance or entering an invalid choice, show the menu again.

---

8. Concepts Used

Variables

A variable stores information.

Example:

```
balance = 1000
```

Functions

A function groups related code into a reusable block.

Example:

```
def show_message():
    print("Welcome")
```

`input()`

Used to receive the user's menu choice.

```
choice = input("Enter your choice: ")
```

`while` loop

Used to repeat the menu.

`if-elif-else`

Used to check which option the user selected.

`break`

Used to stop the loop when the user selects Exit.

---

9. Thought Process

Think about the problem step by step.

Step 1: Store the balance

Create a variable representing the bank balance.

For example:

```
balance = some amount
```

Step 2: Create a menu function

Create a function whose job is to display:

```
1. Check Balance
2. Exit
```

Step 3: Create the main function

Inside the main function, start a loop that keeps running.

Step 4: Display the menu

Call the menu function inside the loop.

This is important because anything inside the loop is repeated.

Step 5: Ask for the user's choice

Store the entered choice in a variable.

Step 6: Check option `1`

If the user enters `1`, show the current balance.

Do not stop the loop.

The menu should appear again.

Step 7: Check option `2`

If the user enters `2`:

1. Print an exit message
2. Use `break`

Step 8: Handle invalid choices

For any other value, print:

```
Invalid choice
```

The loop should then continue automatically.

---

10. Pseudocode

```
START

CREATE a function to display the menu
    PRINT "1. Check Balance"
    PRINT "2. Exit"

CREATE a main function
    SET balance to a starting amount

    WHILE True
        CALL the menu function

        ASK the user to enter a choice

        IF choice is "1"
            PRINT the balance

        ELSE IF choice is "2"
            PRINT exit message
            BREAK the loop

        ELSE
            PRINT invalid choice message

CALL the main function

END
```

---

11. Suggested Solving Approach: Functional Approach

Use two functions.

Function 1: `show_menu()`

Responsibility:

```
Display the available menu options
```

It should not handle the balance or user choice.

Function 2: `main()`

Responsibility:

- Store the balance
- Run the loop
- Call `show_menu()`
- Receive the user's choice
- Check the choice
- Stop the program when required

Suggested structure:

```
def show_menu():
    # Display menu options


<div class="source-blank-line"></div>

def main():
    # Create balance

    while True:
        # Show menu
        # Ask for choice
        # Check choice
        # Use break for exit


<div class="source-blank-line"></div>

main()
```

Complete the missing sections yourself.

---

12. Easy Edge Cases

Edge Case 1: Invalid choice

Input:

```
5
```

Expected behaviour:

```
Invalid choice
```

The menu should appear again.

Edge Case 2: Text instead of a menu number

Input:

```
hello
```

Expected behaviour:

```
Invalid choice
```

The program should not crash.

Edge Case 3: Repeated balance checks

Input:

```
1
1
1
2
```

Expected behaviour:

- Display the balance three times
- Show the menu after every balance check
- Exit only after the user selects 2

Edge Case 4: Immediate exit

Input:

```
2
```

Expected behaviour:

- Print an exit message
- Stop the program immediately

Edge Case 5: Empty input

The user presses Enter without typing anything.

Expected behaviour:

```
Invalid choice
```

Then display the menu again.

---

13. Expected Input

Example interaction:

```
Enter your choice: 1
Enter your choice: 5
Enter your choice: 1
Enter your choice: 2
```

The program receives one choice during each loop repetition.

---

14. Expected Output

Example:

```
1. Check Balance
2. Exit
Enter your choice: 1

Current balance: 1000

1. Check Balance
2. Exit
Enter your choice: 5

Invalid choice

1. Check Balance
2. Exit
Enter your choice: 2

Thank you. Exiting the program.
```

The exact wording can be different, but the behaviour should remain the same.

---

15. Hint Only

Start the repeating section with:

```
while True:
```

Place the menu display and `input()` inside this loop.

For the Exit option, use:

```
break
```

Remember:

- Choosing `1` should not stop the loop.
- Choosing `2` should stop the loop.
- An invalid choice should show an error, then repeat the menu.